NES Emulator

Architecture & 6502 Emulation Reference

This document describes the internal architecture of a cycle-accurate NES emulator written in C# .NET 10 (WinForms). It covers how the 6502 CPU, PPU, APU, cartridge mappers, and front-end all fit together — grounded in the actual source code.

1. Project Structure

The solution contains two projects:

- **NesEmulator.Core** — platform-independent emulator logic: CPU, PPU, APU, cartridge, memory bus, and input.
- **NesEmulator.Desktop** — WinForms front-end: window, GDI+ video rendering, NAudio audio output, and the debugger UI.

2. The Master Clock and the Bus

Everything is driven by a single method — **Bus.Clock()** — which the front-end calls in a tight loop until one video frame is complete:

```
public void Clock()
{
    Ppu.Clock();
    if (SystemClock % 3 == 0)
    {
        Cpu.Clock();
        Apu.Clock();
    }
    if (Ppu.NmiRequested) { Ppu.ClearNmi(); Cpu.NMI(); }
    if (Apu.IrqPending)   Cpu.IRQ();
    SystemClock++;
}
```

The real NES runs at **21.477 MHz** (master clock). The PPU divides that by 4 (~5.37 MHz) and the CPU divides it by 12 (~1.79 MHz). Because 12/4 = 3, the PPU runs exactly **3 ticks for every 1 CPU tick**. The emulator captures this with the % 3 gate: every third master tick, both the CPU and APU execute one cycle; the PPU executes every tick.

RunFrame() loops Clock() until **Ppu.FrameComplete** is set — once every 262 scanlines × 341 PPU ticks = ~89,342 PPU cycles (~29,781 CPU cycles) per frame.

2.1 Memory Map

The 6502 has a 16-bit address space ($0000–$FFFF). Bus.Read/Write maps it as follows:

Address Range	Hardware
$0000–$07FF	2 KB internal RAM (mirrored to $1FFF)
$2000–$3FFF	PPU registers (8 regs, mirrored every 8 bytes)
$4000–$4013	APU channel registers
$4014	OAM DMA — copies 256 bytes of RAM into PPU sprite table

Address Range	Hardware
$4015	APU status read/write
$4016	Controller 1 strobe / read
$4017	Controller 2 read / APU frame counter write
$8000–$FFFF	Cartridge PRG-ROM (bank-switched by mapper)

3. The 6502 CPU

3.1 Registers

Register	Width	Purpose
PC	16-bit	Program Counter — address of the next instruction
A	8-bit	Accumulator — primary arithmetic register
X, Y	8-bit	Index registers — used for address arithmetic
SP	8-bit	Stack Pointer — offset into page $0100–$01FF
P	8-bit	Status register — eight condition flags: N V U B D I Z C

3.2 Status Flags

Flag	Name	Set when…
N	Negative	Result bit 7 is 1
V	Overflow	Signed arithmetic overflows
U	Unused	Always 1
B	Break	Set during a BRK push; not a real hardware flag
D	Decimal	Unused on NES — the 2A03 has no BCD mode
I	IRQ Disable	When set, maskable IRQs are ignored
Z	Zero	Result is 0
C	Carry	Unsigned overflow, or set/cleared by shift instructions

3.3 The Fetch–Decode–Execute Cycle

The CPU uses a **cycle-accurate countdown model**. Clock() decrements a _cycles counter; when it hits zero the next opcode is fetched:

```
public void Clock()
{
    if (_cycles == 0)
    {
        _opcode = Read(PC++);
        ref var instr = ref _lookup[_opcode];
        _cycles = instr.Cycles;

        byte extra1 = instr.AddrMode();   // resolve effective address
        byte extra2 = instr.Operate();    // execute instruction

        _cycles += extra1 & extra2;       // optional page-cross penalty
    }
```

```
        _cycles--;
        TotalCycles++;
    }
```

An instruction with a base cost of 4 cycles causes Clock() to do nothing for the next 3 calls, so the CPU consumes exactly the right number of bus cycles relative to the PPU.

3.4 The Opcode Lookup Table

Cpu6502.Table.cs builds a 256-entry array at construction time. Each entry is a struct holding: the instruction name (used by the disassembler), a delegate to the addressing mode method, a delegate to the instruction method, and the base cycle count. Undefined/illegal opcodes map to XXX(), a no-op stub.

3.5 Addressing Modes

Before an instruction executes, one of 13 addressing mode methods runs to load _addrAbs (effective 16-bit address) or _addrRel (branch offset). Page-crossing adds 1 extra cycle via _cycles += extra1 & extra2.

Mode	Symbol	Description
Implied	IMP	No operand; prefetches A for shift instructions
Accumulator	ACC	Operand is the A register
Immediate	IMM	Operand is the byte following the opcode
Zero Page	ZP0	8-bit address in page $00xx
Zero Page, X	ZPX	8-bit address + X, wraps within page 0
Zero Page, Y	ZPY	8-bit address + Y, wraps within page 0
Relative	REL	Signed 8-bit offset from PC (branches only)
Absolute	ABS	Full 16-bit address
Absolute, X	ABX	16-bit + X; +1 cycle if page crossed
Absolute, Y	ABY	16-bit + Y; +1 cycle if page crossed
Indirect	IND	16-bit pointer; replicates 6502 page-wrap bug
Indexed Indirect	IZX	(zp + X) — pointer in zero page offset by X
Indirect Indexed	IZY	(zp) + Y — pointer in zero page then add Y; +1 if page crossed

The **indirect page-wrap bug** is faithfully emulated in IND(): if the low byte of the pointer is $FF, the high byte is fetched from $xx00 instead of the next page — exactly as the real hardware behaves.

3.6 Instruction Set

Cpu6502.Instructions.cs implements all 56 official instructions:

Group	Instructions
Load / Store	LDA LDX LDY STA STX STY
Transfer	TAX TAY TXA TYA TSX TXS
Stack	PHA PHP PLA PLP
Logical	AND EOR ORA BIT
Arithmetic	ADC SBC CMP CPX CPY
Increment / Decrement	INC INX INY DEC DEX DEY
Shift / Rotate	ASL LSR ROL ROR
Jump / Call	JMP JSR RTS RTI BRK
Branch	BCC BCS BEQ BNE BMI BPL BVC BVS
Flag operations	CLC SEC CLD SED CLI SEI CLV
No-op / Illegal	NOP (XXX stub for illegal opcodes)

ADC / SBC: SBC is implemented by XOR-inverting the operand (value ^ 0xFF) and reusing ADC's logic — subtraction with borrow is identical to addition with the inverted operand when the carry flag acts as 'borrow inverted'. The overflow flag V uses the formula: overflow occurs when both inputs have the same sign but the result has a different sign.

Branches add 1 cycle if taken, and another if the destination crosses a page boundary — handled entirely inside BranchIf().

3.7 Interrupts

Interrupt	Vector	Maskable?	Description
Reset	$FFFC / $FFFD	No	Reads reset vector; sets SP=$FD. Called on cartridge insert.
NMI	$FFFA / $FFFB	No	Pushes PC+P, jumps to vector. Fired by PPU at VBlank each frame.
IRQ	$FFFE / $FFFF	Yes (I flag)	Same push sequence. Fired by APU frame counter and DMC channel.

4. The PPU (2C02)

The PPU renders 262 scanlines per frame at 341 PPU cycles per scanline:

Scanlines	Role
–1	Pre-render: clears VBlank/sprite-zero/overflow flags; reloads Y scroll
0–239	Visible: pixel output
240	Idle
241	Sets VBlank flag; fires NMI to CPU if PPUCTRL bit 7 is set
242–260	VBlank period

4.1 The Loopy Scroll Mechanism

The PPU maintains two 15-bit internal registers (_**v** and _**t**), a 3-bit fine-X offset (_**x**), and a write-latch (_**w**). The bits of _v encode five fields simultaneously:

```
_v:  yyy NN YYYYY XXXXX
      |  ||   |     +----- Coarse X: tile column 0-31
      |  ||   +----------- Coarse Y: tile row 0-29
      |  |+--------------- Nametable select (X/Y)
      |  +---------------- Fine Y: pixel row within tile 0-7
```

Writes to $2005 (PPUSCROLL) and $2006 (PPUADDR) manipulate _**t** and _**w**. At cycle 257 of each scanline the X portion of _t is copied into _v; during the pre-render scanline (cycles 280–304) the Y portion is copied. This is what makes smooth horizontal and vertical scrolling work.

4.2 Background Rendering Pipeline

Every 8 PPU cycles the pipeline fetches 4 bytes for the next tile column:

- **Nametable byte** — tile ID from $2000 + (v & $0FFF)
- **Attribute byte** — 2-bit palette selector from $23C0 + ...
- **Pattern low byte** — bit-plane 0 of the tile from CHR-ROM
- **Pattern high byte** — bit-plane 1 of the tile from CHR-ROM

These are loaded into four 16-bit shift registers. Each cycle the registers shift left by 1, and the bits that fall off — selected by fine-X — form the 2-bit pixel index and 2-bit palette index for the current dot.

4.3 Sprite Rendering

At cycle 257 of each scanline, **EvaluateSprites()** scans all 64 OAM entries and copies up to 8 visible sprites into secondary OAM. At cycle 340, **FetchSpriteTiles()** loads their pattern data into 8 pairs of shift registers, handling both 8×8 and 8×16 sprites and horizontal/vertical flipping.

During pixel output, sprite shift registers are scanned left-to-right; the first non-transparent pixel wins. Priority (behind / in-front-of background) is controlled by bit 5 of each sprite's attribute byte.

Sprite-zero hit is set when both the background and the first OAM entry produce non-transparent pixels at the same dot. Games use this as a raster timing signal to split the screen mid-frame.

4.4 Nametable Mirroring

The NES has 2 KB of VRAM but addresses 4 nametables. NtIndex() maps a nametable address to physical VRAM bank 0 or 1:

Mirror Mode	Mapping
Horizontal	NT 0 and 1 → bank 0; NT 2 and 3 → bank 1 (vertical scrolling)
Vertical	NT 0 and 2 → bank 0; NT 1 and 3 → bank 1 (horizontal scrolling)
Single Low	All NTs → bank 0
Single High	All NTs → bank 1

Mapper 1 (MMC1) changes mirror mode dynamically via its control register, enabling four-screen scrolling effects.

5. The APU (2A03)

The APU generates audio at ~1.789 MHz and downsamples to 44,100 Hz. Every CPU cycle, Apu2A03.Clock() advances the channels, then accumulates a fractional counter to know when to emit a sample:

```
_sampleAccum += 44100 / 1_789_773.0;   // ~0.02465 per CPU cycle
if (_sampleAccum >= 1.0)
{
    _sampleAccum -= 1.0;
    Samples.Add(Mix());                 // ~735 samples/frame at 60 fps
}
```

After RunFrame() the front-end calls NesAudioProvider.Submit(), which converts the float samples to IEEE bytes and pushes them into NAudio's BufferedWaveProvider (80 ms target latency).

5.1 Channels

Channel	Waveform	Key Mechanism
Pulse 1 & 2	Square wave	4 selectable duty cycles (12.5/25/50/75%); sweep unit bends pitch each half-frame
Triangle	Triangle wave	32-step sequencer; silenced if timer period < 2 (ultrasonic)
Noise	White noise	15-bit LFSR; short mode (bit 6 feedback) produces metallic tones
DMC	1-bit delta	Reads raw sample bytes from cartridge ROM at $C000+; output climbs/falls by 2 per bit

5.2 Frame Counter

The frame counter steps at CPU cycles 3729, 7457, 11186, and 14915 (4-step mode). Each step clocks:

- **Quarter frame** — advances envelope decay and the triangle's linear counter
- **Half frame** — advances length counters (note durations) and sweep units

5-step mode adds a fifth step at cycle 18641 and disables the frame IRQ. Written by the game to $4017.

5.3 Mixing

The five channel outputs are combined using the NES's non-linear mixing formula, approximated linearly. Coefficients are derived from the real NES lookup-table mixing curves:

```
float pulse = 0.00752f * (p1 + p2);
float tnd   = 0.00851f * tri + 0.00494f * noise + 0.00335f * dmc;
return pulse + tnd;   // output range approximately 0.0-0.9
```

6. Cartridges and Mappers

iNES ROM files begin with a 16-byte header identifying the number of 16 KB PRG-ROM banks, 8 KB CHR-ROM banks, the mapper number, and the nametable mirror mode. Cartridge.Load() parses this and instantiates the correct mapper via a switch expression.

6.1 Mapper 0 — NROM

No banking. A 16 KB ROM mirrors itself to fill $8000–$FFFF; a 32 KB ROM maps directly. Read-only. Covers Donkey Kong, Pac-Man, Super Mario Bros. 1, and roughly 10% of the library.

6.2 Mapper 1 — MMC1

Uses a 5-bit serial shift register. Writing bit 7 of any $8000–$FFFF address resets it. After 5 consecutive writes the accumulated value is loaded into one of four internal registers based on the address range written:

Address	Register	Controls
$8000–$9FFF	Control	Mirror mode (2 bits), PRG banking mode (2 bits), CHR banking mode (1 bit)
$A000–$BFFF	CHR Bank 0	4 KB or 8 KB CHR bank at PPU $0000
$C000–$DFFF	CHR Bank 1	4 KB CHR bank at PPU $1000 (4 KB mode only)
$E000–$FFFF	PRG Bank	16 KB PRG bank number (bits 0–3); PRG-RAM disable (bit 4)

PRG banking supports three modes: 32 KB switchable, fix first 16 KB / switch last, or switch first / fix last. CHR banking supports 8 KB or dual 4 KB modes. Games with no CHR-ROM (e.g. Zelda, Metroid) use 8 KB of CHR-RAM allocated internally by the mapper. Covers ~28% of the library.

7. Controllers

Controller holds a single shift register byte. On strobe (Write(1) then Write(0)), the current button state is latched. Each Read() returns bit 0 and shifts right, so the CPU reads buttons in order:

Bit	Button
0	A
1	B
2	Select
3	Start
4	Up
5	Down
6	Left
7	Right

The front-end maps keyboard keys to Controller.SetButton() calls in MainForm.SetKey():

Key	NES Button
Z	A
X	B
Right Shift	Select
Enter	Start
Arrow keys	Up / Down / Left / Right

8. Front-End (WinForms)

MainForm owns a 16 ms System.Windows.Forms.Timer (~60 fps). Each tick:

- **Emulate** — Bus.RunFrame() executes ~29,781 CPU cycles, producing one complete video frame and ~735 audio samples.
- **Audio** — NesAudioProvider.Submit() drains the APU sample list into NAudio's ring buffer (300 ms buffer, 80 ms target latency, overflow silently discarded).
- **Video** — BlitFrame() copies Ppu.FrameBuffer (a uint[] of ARGB pixels) into a 256x240 Bitmap via Marshal.Copy and assigns it to a PictureBox with SizeMode=Zoom.
- **Debug UI** — UpdateUi() refreshes registers, flags, stack, disassembly, and memory dump panels.

8.1 Debugger

The debugger panel disassembles $8000–$FFFF on ROM load using Cpu6502.Disassemble(), which walks the address range interpreting bytes as instructions and formatting operands. The currently-executing address is highlighted in blue via owner-draw on the ListBox.

Step modes: **Step Instruction (F7)** advances one CPU instruction at a time; **Step Frame (F8)** advances exactly one video frame. **Run (F5)** starts the 60 fps timer; **Stop (F6)** halts it.